

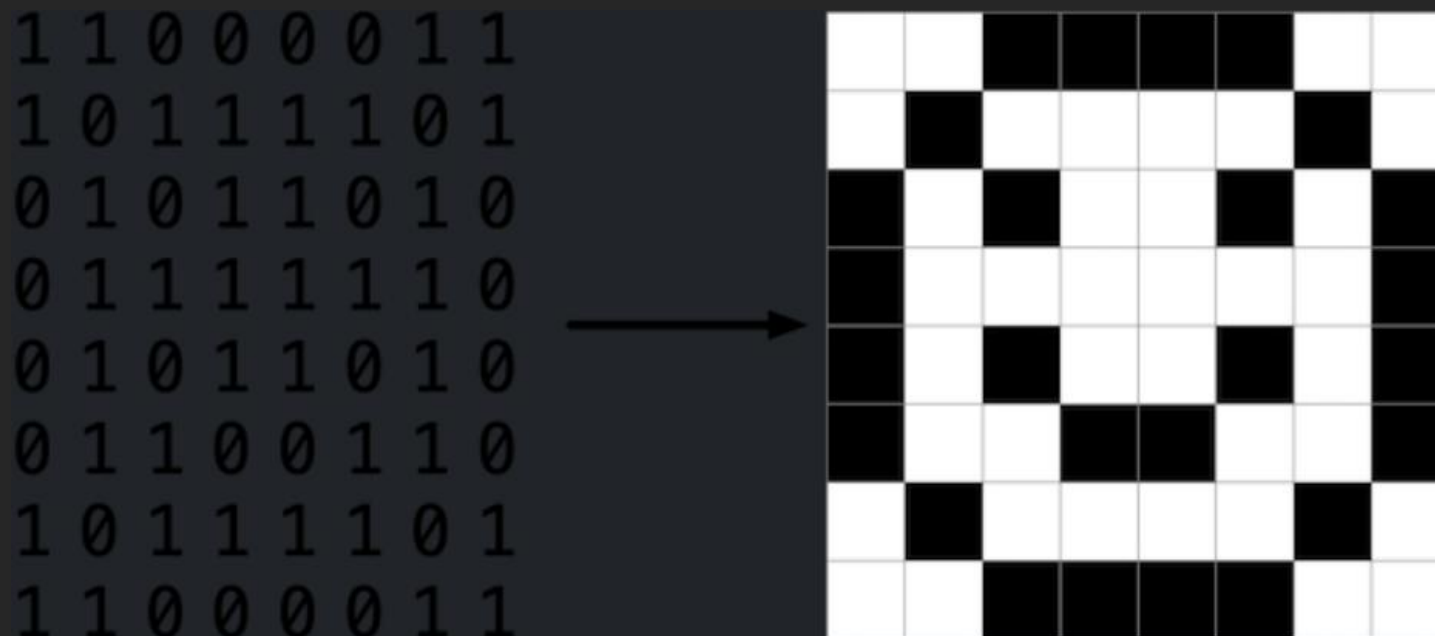


Filter-less

Problem set



احتمالا ساده‌ترین روش برای نشان دادن یک عکس، با یک جدول از پیکسل‌ها باشد که هرکدام از آنها می‌توانند یک رنگ متفاوت باشند. برای عکس‌های سیاه و سفید ما به یک بیت برای هر پیکسل نیاز داریم: 0 می‌تواند سیاه و 1 می‌تواند سفید را نشان دهد.



در اینجا، این یک نقشه از بیت‌ها است. برای عکس‌های رنگی، باید تعداد بیت‌های بیشتری به ازای هر پیکسل در نظر بگیریم. انواع فایل‌های BMP، JPEG و PNG، که “24-bit color” هستند، از 24 بیت به ازای هر پیکسل استفاده می‌کنند. (BMP در واقع از رنگ‌های ۱، ۴، ۸، ۱۶، ۲۴ و ۳۲ بیتی پشتیبانی می‌کند.)

یک BMP با پیکسل‌های 24 بیتی، از 8 بیت برای نشان دادن مقدار رنگ قرمز در هر پیکسل، 8 بیت برای نشان دادن سبز و 8 بیت برای نشان دادن آبی استفاده می‌کند. اینها رنگ‌های RGB هستند.

اگر مقادیر R، G و B در یک پیکسل در مبنای عدد نویسی مبنای 16 به صورت 0xff، 0x00 و 0x00 باشند، این پیکسل کاملاً قرمز است، زیرا 0xff (یا 255 در مبنای ده)، به معنی مقدار زیادی قرمز است و 0x00 برای دو رنگ بعدی، به معنی هیچ سبز و هیچ آبی است. در این مسئله شما با تغییر دادن مقادیر R، G و B در هر پیکسل، فیلترهای خودتان را می‌سازید.

در فایل به اسم `helpers.c` در فولدری به اسم `filter-less`، برنامه‌ای بنویسید که به تصاویر BMP فیلتر اعمال کند.

برای این مسئله، شما باید عملکرد کدی که توسط تیم CS50 در اختیار شما قرار گرفته را ارتقا دهید.

Distribution code را دانلود کنید.

در <https://cs50.dev> وارد شوید، روی ترمینال کلیک کرده و `cd` را به تنهایی اجرا کنید، ترمینال شما باید شبیه نمونه زیر باشد:

```
$
```

حالا دستور زیر را اجرا کنید تا فایل `filter-less.zip` به اسم `filter-less.zip` را دانلود کنید:

```
wget https://cdn.cs50.net/2024/fall/psets/4/filter-less.zip
```

حالا با اجرا کردن دستور زیر فولد ری به اسم `filter-less` بسازید:

```
unzip filter-less.zip
```

از حالا به بعد دیگر نیازی به فایل zip ندارید پس با دستور زیر به همراه Y آن را حذف کنید:

```
rm filter-less.zip
```

حالا دستور زیر را اجرا کنید تا به فولدر مورد نظر منتقل شوید:

```
cd filter-less
```

ترمینال شما باید شبیه نمونه زیر باشد:

```
filter-less/ $
```

حالا با اجرا کردن دستور ls باید فایل‌های: `helpers.h`، `helpers.c`، `filter.c`، `Makefile` و `images` را مشاهده کنید. همچنین باید فولدری به اسم `images` که فایل‌های BMP را شامل می‌شود هم ببینید. اگر به مشکلی برخوردید به گام‌های بالا برگردید و سعی کنید جایی که اشتباه کردید را بفهمید.

نقشه بیت، کمی فنی‌تر

به یاد داشته باشید که یک فایل در واقع یک دنباله از بیت‌ها است که به شکل‌های مختلف مرتب شده‌اند. یک فایل BMP با پیکسل‌های 24 بیتی هم یک دنباله از بیت‌هاست که تقریباً هر 24 بیت آن رنگ یک پیکسل را نشان می‌دهد. یک فایل BMP مقداری metadata را هم شامل می‌شود؛ مثل طول و عرض عکس. این metadata در ابتدای فایل به شکل دو ساختمان داده به اسم header ذخیره می‌شوند و این Headerها نباید با headerهای C اشتباه گرفته شوند. (لازم به ذکر است که این هدرها در طول زمان تکامل یافته‌اند. در اینجا از جدیدترین نسخه‌ی فرمت BMP مایکروسافت، یعنی نسخه ۴.۰، که همراه با ویندوز ۹۵ معرفی شد، استفاده می‌کنیم.)

اولین هدر، که BITMAPFILEHEADER نام دارد، ۱۴ بایت طول دارد. (به یاد داشته باشید که ۱ بایت برابر با ۸ بیت است.) هدر دوم، که BITMAPINFOHEADER نام دارد، ۴۰ بایت طول دارد.

بلافاصله پس از این هدرها، بیت‌مپ واقعی قرار دارد: آرایه‌ای از بایت‌ها که هر سه بایت آن نشان‌دهنده‌ی رنگ یک پیکسل است. با این حال، BMP این سه‌تایی‌ها را برعکس ذخیره می‌کند (یعنی به صورت BGR)، به این ترتیب که ۸ بیت برای رنگ آبی، سپس ۸ بیت برای رنگ سبز و در نهایت ۸ بیت برای رنگ قرمز اختصاص داده می‌شود.

بلافاصله پس از این هدرها، بیت‌مپ واقعی قرار دارد: آرایه‌ای از بایت‌ها که هر سه بایت آن نشان‌دهنده‌ی رنگ یک پیکسل است. با این حال، BMP این سه‌تایی‌ها را برعکس ذخیره می‌کند (یعنی به صورت BGR)، به این ترتیب که ۸ بیت برای رنگ آبی، سپس ۸ بیت برای رنگ سبز و در نهایت ۸ بیت برای رنگ قرمز اختصاص داده می‌شود.

برخی از فایل‌های BMP کل تصویر را نیز وارونه ذخیره می‌کنند، به طوری که ردیف بالای تصویر در انتهای فایل BMP قرار می‌گیرد. اما در این مجموعه مسائل، فایل‌های BMP به همان روشی که توضیح داده شد ذخیره شده‌اند، یعنی ردیف بالای تصویر در ابتدا و ردیف پایین در انتها قرار دارد.

به عبارت دیگر، اگر بخواهیم شکلک ۱ بیتی بالا را به یک شکلک ۲۴ بیتی تبدیل کنیم و رنگ قرمز را جایگزین مشکی کنیم، یک فایل بیت‌مپ 24 بیتی این بیت‌مپ را به صورت زیر ذخیره می‌کند. در اینجا، 0000ff نشان‌دهنده‌ی قرمز و ffffff نشان‌دهنده‌ی سفید است؛ تمام موارد 0000ff در تصویر با رنگ قرمز مشخص شده‌اند

```
ffffff fffffff 0000ff 0000ff 0000ff 0000ff fffffff fffffff
ffffff 0000ff fffffff fffffff fffffff fffffff 0000ff fffffff
0000ff fffffff 0000ff fffffff fffffff 0000ff fffffff 0000ff
0000ff fffffff fffffff fffffff fffffff fffffff fffffff 0000ff
0000ff fffffff 0000ff fffffff fffffff 0000ff fffffff 0000ff
0000ff fffffff fffffff 0000ff 0000ff fffffff fffffff 0000ff
ffffff 0000ff fffffff fffffff fffffff fffffff 0000ff fffffff
ffffff fffffff 0000ff 0000ff 0000ff 0000ff fffffff fffffff
```

با توجه به اینکه ما این بیت‌ها را از چپ به راست و بالا به پایین نشان داده‌ایم، می‌توانیم یک شکلک خندان را مشاهده کنیم!

به یاد داشته باشید که یک رقم مبنای 16، با 4 بیت نشان داده می‌شود. به همین ترتیب، `ffffff` در باینری به شکل `11111111111111111111` است.

توجه داشته باشید که می‌توانید یک نقشه بیت را به عنوان یک آرایه دو بعدی از پیکسل‌ها نشان دهید: یک عکس، یک آرایه از ردیف‌ها و هر ردیف یک آرایه از پیکسل‌هاست. این همانطوری است که ما تصمیم گرفتیم نقشه بیت‌ها را در این مسئله نشان دهیم.

فیلتر کردن عکس دقیقا به چه معناست؟ می‌توانید تغییر دادن پیکسل‌های یک عکس به گونه‌ای که تاثیر آن در عکس نهایی محسوس باشد را فیلتر کردن عکس بنامید.



حال بیایید به چند فایلی که در اختیار شما قرار داده شده است نگاهی بیاندازیم.

`bmp.h`

فایل `bmp.h` را با دوبار کلیک کردن باز کنید.

در اینجا تعاریف هدرهایی که قبلاً ذکر کردیم (`BITMAPINFOHEADER` و `BITMAPFILEHEADER`) را مشاهده

خواهید کرد. علاوه بر این، انواع داده‌های `BYTE`، `DWORD`، `LONG` و `WORD` را نیز تعریف می‌کند که معمولاً در برنامه‌نویسی ویندوز استفاده می‌شوند.

توجه کنید که این‌ها فقط نام‌های مستعاری برای انواع داده‌های پایه هستند که احتمالاً از قبل با آن‌ها آشنا هستید.

به نظر می‌رسد که `BITMAPFILEHEADER` و `BITMAPINFOHEADER` از این انواع داده استفاده می‌کنند.

شاید مهم‌ترین نکته برای شما این باشد که این فایل همچنین یک `struct` به نام `RGBTRIPLE` تعریف می‌کند که به سادگی سه بایت را «کپسوله» می‌کند: یکی برای آبی، یکی برای سبز و یکی برای قرمز (ترتیبی که باید در آن `RGB triple`ها را در فایل دیسک پیدا کنیم).

چرا این ساختمان‌های داده مفید هستند؟ خوب، به یاد داشته باشید که یک فایل فقط دنباله‌ای از بایت‌ها (یا در نهایت، بیت‌ها) روی دیسک است. اما این بایت‌ها معمولاً به گونه‌ای مرتب شده‌اند که چند بایت اول چیزی را نشان می‌دهند، چند بایت بعدی چیز دیگری را نمایش می‌دهند و همین‌طور الی آخر.

"فرمت‌های فایل" وجود دارند زیرا استانداردهایی تعیین شده‌اند که هر بایت چه معنایی داشته باشد. حال، می‌توانیم یک فایل را مستقیماً از دیسک به عنوان یک آرایه‌ی بزرگ از بایت‌ها در حافظه بارگذاری کنیم و صرفاً به خاطر بسپاریم که بایت در `array[i]` یک چیز را نشان می‌دهد، در حالی که بایت در `array[j]` چیز دیگری را.

اما چرا به برخی از این بایت‌ها نام ندهیم تا بتوانیم آن‌ها را راحت‌تر از حافظه بازیابی کنیم؟ این دقیقاً همان کاری است که ساختارهای تعریف‌شده در `bmp.h` انجام می‌دهند. به جای اینکه یک فایل را به عنوان یک دنباله‌ی طولانی از بایت‌ها در نظر بگیریم، می‌توانیم آن را به عنوان دنباله‌ای از ساختارها ببینیم.

`filter.c`

حالا وقت این است که `filter.c` را باز کنید. با اینکه این فایل از قبل نوشته شده است، چند نکته مهم را باید بدانید. اولاً، به تعریف فیلترها در خط 10 توجه کنید. این رشته به برنامه می‌گوید که آرگومان‌های مجاز خط فرمان چیست: `b`، `e`، `g`، و `r`. هرکدام از این‌ها یک فیلتر مختلف که ممکن است به تصاویر خود اعمال کنیم را مشخص می‌کنند: `blur`، `edge detection`، `grayscale` و `reflection`. خطوط بعدی یک فایل تصویری را باز می‌کنند، از BMP بودن آن مطمئن می‌شوند و تمام اطلاعات پیکسل‌ها را خوانده و در یک آرایه‌ی دو بعدی به اسم `image` می‌نویسند. به پایین بروید تا به دستور `switch` برسید که از خط 101 آغاز می‌شود. توجه کنید که بسته به فیلتر انتخابی، توابع مختلفی فراخوانی می‌شود: اگر کاربر فیلتر `b` را انتخاب کند، برنامه تابع `blur` را فراخوانی می‌کند؛ اگر `e` را انتخاب کند، تابع `edges` فراخوانی می‌شود؛ اگر `g` را انتخاب کند، تابع `grayscale` فراخوانی می‌شود؛ و اگر `r` را انتخاب کند، تابع `reflect` فراخوانی می‌شود. همچنین توجه کنید که هرکدام از این توابع به عنوان آرگومان، ارتفاع تصویر، عرض تصویر و آرایه دو بعدی پیکسل‌ها را دریافت می‌کنند. اینها توابعی هستند که شما باید در آنها آرایه دو بعدی پیکسل‌ها را به گونه‌ای ویرایش کنید که فیلتر مورد نظر روی تصویر نهایی اعمال شود. بقیه خطوط این فایل، `image` حاصل را گرفته و آن را در یک فایل تصویری جدید می‌نویسد.

helpers.h

فایل کوتاهی است که تعریف توابعی که در قسمت قبل دیدید را شامل می‌شود.

در اینجا توجه کنید که هر تابع یک آرایه‌ی دوبعدی به نام `image` را به عنوان آرگومان دریافت می‌کند. این آرایه شامل تعدادی ردیف به اندازه‌ی `height` است، و هر ردیف، خود یک آرایه‌ی دیگر از تعدادی `RGBTRIPLE` به اندازه‌ی `width` می‌باشد.

بنابراین، اگر `image` کل تصویر را نمایش دهد، آنگاه `image[0]` نشان‌دهنده‌ی اولین ردیف است و `image[0][0]` نشان‌دهنده‌ی پیکسلی در گوشه‌ی بالا-چپ تصویر خواهد بود.

helpers.c

در فایل `helpers.c` می‌توانید پیاده‌سازی توابعی که در `helpers.h` دیدیم را مشاهده کنیم. همانطور که می‌بینید، کامل کردن این فایل به عهده شماست!

Makefile

این فایل مشخص می‌کند که وقتی دستوراتی از قبیل `makefilter` را در ترمینال اجرا می‌کنیم چه اتفاقی باید بیافتد. مسائل قبلی که تا اینجا حل کرده‌اید تک فایلی بودند، اما مسئله `filter` دارای چند فایل است: `filter.c` و `helpers.c`. باید به دستور `make` نحوه کامپایل کردن این فایل را بگوییم.

سعی کنید `filter` را با دستور زیر کامپایل کنید:

```
$ make filter
```

سپس می‌توانید با دستور زیر برنامه را اجرا کنید:

```
$ ./filter -g images/yard.bmp out.bmp
```

که یک تصویر را از `images/yard.bmp` برمی‌دارد و بعد از اعمال تابع `grayscale` روی تمام پیکسل‌ها، یک تصویر جدید به اسم `out.bmp` می‌سازد. این تابع هنوز کار خاصی انجام نمی‌دهد پس تصویر نهایی باید شبیه تصویر اصلی باشد.

توابع `helpers.c` را طوری پیاده‌سازی کنید که کاربر بتواند روی تصاویرش فیلترهای: `blur`، `reflection`، `grayscale` یا `edge` را اعمال کند.

- تابع `grayscale` باید تصویر را بگیرد و آن را به نسخه سیاه-سفیدش تبدیل کند.
- تابع `reflect` باید تصویر را بگیرد و آن را به صورت افقی از وسط آینه‌ای کند.
- تابع `blur` باید تصویر را بگیرد و آنرا به نسخه `box-blur` خود تبدیل کند.
- تابع `edges` باید تصویر را بگیرد و لبه‌های بین اشیا را بر اساس عملگر سوبل پررنگ کند.

نباید امضای هیچ‌یک از توابع را تغییر دهید و همچنین نباید هیچ فایل دیگری غیر از `helpers.c` را ویرایش کنید.

Grayscale را پیاده‌سازی کنید

یکی از فیلترهای رایج، فیلتر "grayscale" است که در آن یک تصویر را گرفته و به سیاه و سفید تبدیل می‌کنیم. این فرآیند چگونه کار می‌کند؟

- به یاد داشته باشید که اگر قرمز، سبز و آبی همه `0x00` باشند، آن پیکسل سیاه خواهد بود. و اگر تمام مقادیر `0xff` باشند، آن پیکسل سفید خواهد بود. تا زمانی که مقادیر قرمز، سبز و آبی برابر باشند، نتیجه، پرده‌های مختلفی از خاکستری در امتداد طیف سیاه و سفید خواهد بود، به طوری که مقادیر بالاتر به معنای پرده‌های روشن‌تر (نزدیک‌تر به سفید) و مقادیر پایین‌تر به معنای پرده‌های تیره‌تر (نزدیک‌تر به سیاه) هستند.
- پس برای تبدیل یک پیکسل به حالت پرده‌های خاکستری، فقط کافی است که مقادیر قرمز، سبز و آبی را برابر قرار دهید. اما چگونه می‌توان مقدار مناسبی برای آن‌ها تعیین کرد؟ احتمالاً منطقی است که اگر مقادیر اولیه‌ی قرمز، سبز و آبی همگی نسبتاً بالا باشند، مقدار جدید نیز باید بالا باشد و اگر مقادیر اولیه همگی پایین باشند، مقدار جدید نیز باید پایین باشد.
- در واقع، برای اطمینان از اینکه هر پیکسل در تصویر جدید همچنان همان میزان روشنایی یا تیرگی کلی تصویر اصلی را حفظ کند، می‌توان میانگین مقادیر قرمز، سبز و آبی را گرفت تا پرده خاکستری مناسب برای پیکسل جدید تعیین شود

```
void grayscale(int height, int width, RGBTRIPLE  
image[height][width]) {
```

```
    // Loop over all pixels
```

```
    // Take average of red, green, and blue
```

```
    // Update pixel values
```

```
}
```

اگر الگوریتم بالا را روی هر پیکسل در تصویر اعمال کنید، نتیجه یک تصویر پرده خاکستری خواهد بود. بیایید مقداری شبه کد برای حل این مسئله بنویسیم:

```
void grayscale(int height, int width, RGBTRIPLE image[height][width]) {
```

```
    // Loop over all pixels
```

```
    for (int i = 0; i < height; i++)
```

```
    {
```

```
        for (int j = 0; j < width; j++)
```

```
        {
```

```
            // Take average of red, green, and blue
```

```
            // Update pixel values
```

```
        }
```

```
    }
```

```
}
```

چگونه می‌خواهید درون برنامه روی همه‌ی پیکسل‌ها حلقه بزنید؟ به یاد داشته باشید که پیکسل‌های تصاویر در یک آرایه دو بعدی، `image`، ذخیره شده‌اند. برای حرکت روی یک آرایه دو بعدی شما به دو حلقه نیاز دارید که یکی درون دیگری باشد.

حال، شما می‌توانید از `image[i][j]` برای دسترسی به هر پیکسل تصویر استفاده کنید. اما چگونه می‌توان میانگین مقادیر قرمز، سبز و آبی را محاسبه کرد؟ به یاد داشته باشید که هر عنصر از `image` یک `RGBTRIPLE` است، که ساختاری است که در `bmp.h` برای نمایش یک پیکسل تعریف شده است. برای دسترسی به اعضای یک `struct`، روش معمول این کار استفاده می‌شود: `image[i][j].rgbtRed` به شما دسترسی به مقدار قرمز `RGBTRIPLE` را می‌دهد، `image[i][j].rgbtGreen` به شما دسترسی به مقدار سبز آن می‌دهد و

زمانی که میانگین را محاسبه می‌کنید، به یاد داشته باشید که مقادیر `rgbtRed`، `rgbtGreen` و `rgbtBlue` همه اعداد صحیح هستند. پس مطمئن شوید که هر عدد اعشاری را هنگام تخصیص آن به مقدار پیکسل، به نزدیک‌ترین عدد صحیح گرد کنید! به نظرتان چرا مجموع این اعداد صحیح را باید بر 3.0 تقسیم کنیم و نه بر 3؟

پس از اینکه مقادیر قرمز، سبز و آبی پیکسل را به یک رنگ خاکستری میانگین تبدیل کردید، می‌توانید مقادیر قرمز، سبز و آبی پیکسل را به‌روزرسانی کنید. تا این مرحله، شما با ساختار نوشتاری `assignment` آشنا شده‌اید!

Sepia را پیاده‌سازی کنید

بیشتر برنامه‌های ویرایش عکس از فیلتر “sepia” پشتیبانی می‌کنند که با قرمز-قهوه‌ای کردن تصویر، به آن حس و حال قدیمی می‌دهند.

- یک تصویر می‌تواند با محاسبه مقادیر جدید قرمز، سبز و آبی (با توجه به مقادیر اولیه) در هر پیکسل، به sepia تبدیل شود.
- چندین الگوریتم برای تبدیل یک تصویر به sepia وجود دارد؛ اما برای این مسئله ما الگوریتم زیر را پیشنهاد می‌کنیم. برای هر پیکسل، با توجه به مقادیر اصلی رنگ‌ها، مقادیر رنگی sepia جدید مطابق زیر محاسبه شود:

```
sepiaRed = .393 * originalRed + .769 * originalGreen + .189 * originalBlue
sepiaGreen = .349 * originalRed + .686 * originalGreen + .168 * originalBlue
sepiaBlue = .272 * originalRed + .534 * originalGreen + .131 * originalBlue
```

- البته، نتیجه هرکدام از این فرمول‌ها ممکن است عددی اعشاری باشد که باید به نزدیک‌ترین عدد صحیح گرد شود. همچنین ممکن است نتیجه فرمول عددی بزرگتر از 255، حداکثر مقدار برای یک رنگ 8 بیتی، باشد. در این صورت، مقادیر قرمز، سبز و آبی باید در 255 محدود شوند. به این ترتیب، می‌توانیم تضمین کنیم که مقادیر قرمز، سبز و آبی نهایی عددهای صحیحی بین 0 و 255 خواهند بود، از جمله 0 و 255.

مقداری شبه کد بنویسید و به یاد داشته باشید که با حلقه `for` به صورت یکی درون دیگری می‌توانید به هر پیکسل دسترسی پیدا کنید

```
void sepia(int height, int width, RGBTRIPLE image[height][width])
{
    // Loop over all pixels
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
        {
            // Compute sepia values

            // Update pixel with sepia values
        }
    }
}
```

برای محاسبه مقادیر `sepia` به قدم‌های بالا مراجعه کنید. درست است که محاسبه `sepia` فرمول دارد اما چند نکته دیگر هم حائز اهمیت می‌باشند:

- نتیجه هر محاسبه را به نزدیک‌ترین عدد صحیح گرد کنید.
- مطمئن شوید که عدد نهایی از 255 بزرگتر نباشد.

یک تابع که کمترین از دو عدد صحیح را برمی‌گرداند چگونه می‌تواند در پیاده‌سازی فیلتر `sepia` مفید باشد، به‌ویژه زمانی که نیاز دارید مطمئن شوید مقدار رنگ از 255 بیشتر نمی‌شود؟ می‌توانید یک تابع کمکی برای انجام همین کار بنویسید، اما این کار الزامی نیست!

Reflect را پیاده‌سازی کنید

برخی فیلترها ممکن است پیکسل‌ها را جابه‌جا کنند. استفاده از فیلتر `reflect` مثل آن است که تصویر را روبه‌روی یک آینه قرار بدهیم.

- تمام پیکسل‌های سمت چپ باید به سمت راست انتقال پیدا کنند و برعکس.
- دقت کنید که تمام پیکسل‌های تصویر اصلی باید در تصویر نهایی باشند و تنها جای آنها ممکن است عوض شده باشند.

```
void reflect(int height, int width, RGBTRIPLE image[height][width])
{
    // Loop over all pixels
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
        {
            // Swap pixels
        }
    }
}
```

در تابع `reflect`، باید مکان پیکسل‌ها را به مکان مخالف در ردیف خود (نسبت به وسط) انتقال دهید. برای این کار اول مقداری شبه کد بنویسید:

برای تعویض دو مقدار با هم، استفاده از یک متغیر موقت را به یاد آورید؛ پس هیچ نیازی به یک تابع جداگانه برای این جابه‌جایی نیست.

حالا زمان خوبی برای فکر کردن درباره حلقه‌های تو در تو `for` است. حلقه بیرونی روی هر ردیف و حلقه درونی روی پیکسل‌های این ردیف حرکت می‌کند. به نظرتان برای بازتاب یک ردیف نیاز است روی تمام پیکسل‌های آن حرکت کرد؟

Blur (محو) را پیاده‌سازی کنید

روش‌های زیادی برای ایجاد اثر محو یا نرم کردن یک تصویر وجود دارد. برای این مسئله، ما از "box-blur" استفاده خواهیم کرد که به این صورت عمل می‌کند: هر پیکسل را گرفته و برای هر مقدار رنگی، مقدار جدیدی به آن اختصاص می‌دهد که میانگین مقادیر رنگی پیکسل‌های مجاور است.

- شبکه پیکسلی زیر را در نظر بگیرید که به هر پیکسل یک عدد اختصاص داده‌ایم.

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

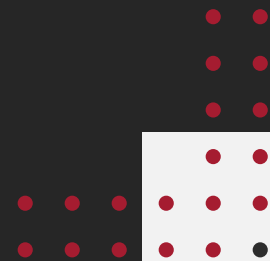
- مقدار جدید هر پیکسل به این صورت خواهد بود که میانگین مقادیر تمام پیکسل‌هایی که در یک ردیف و ستون از پیکسل اصلی قرار دارند (که یک محدوده 3×3 تشکیل می‌دهند) محاسبه شود. به عنوان مثال، هر یک از مقادیر رنگی پیکسل 6 با میانگین مقادیر رنگی پیکسل‌های 1، 2، 3، 5، 6، 7، 9، 10 و 11 به دست می‌آید (توجه داشته باشید که خود پیکسل 6 نیز در میانگین محاسبه می‌شود). به همین ترتیب، مقادیر رنگی برای پیکسل 11 با میانگین مقادیر رنگی پیکسل‌های 6، 7، 8، 10، 11، 12، 14، 15 و 16 به دست می‌آید.
- برای یک پیکسل در حاشیه یا گوشه، مانند پیکسل 15، باز هم به دنبال تمام پیکسل‌ها در یک ردیف و ستون خواهیم بود؛ در این مورد، پیکسل‌های 10، 11، 12، 14، 15 و 16 را در نظر می‌گیریم.

```
void blur(int height, int width, RGBTRIPLE image[height][width])
{
    // Create a copy of image
    RGBTRIPLE copy[height][width];
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
        {
            copy[i][j] = image[i][j];
        }
    }
}
```

هنگام پیاده‌سازی تابع `blur`، ممکن است متوجه شوید که محو کردن یک پیکسل باعث تغییر محو شدن پیکسل دیگری می‌شود. بهتر است یک کپی از آرایه تصویر اولیه ایجاد کنید. این کار با اعلان یک آرایه دو بعدی جدید مانند `copy` با استفاده از دستور `RGBTRIPLE copy[height][width];` انجام می‌گیرد. سپس تصویر اصلی را با استفاده از حلقه‌های تو در تو، پیکسل به پیکسل، در آرایه `copy` کپی کنید:

حالا می‌توانید رنگ پیکسل‌ها را از `copy` بخوانید نتیجه محاسبه را در `image` بنویسید (تغییر) دهید

تمام فیلترها را روی فایل‌های نمونه‌ی نقشه بیت تست کنید.



درستی

```
check50 cs50/problems/2025/x/filter/less
```

استایل

```
style50 helpers.c
```

نحوه ارسال

```
submit50 cs50/problems/2025/x/filter/less
```

CS50x Iran

Harvard's Computer Science 50x Iran

